Nike

Product Copy App

Description: The app is a product copy generator. This info goes to the nike.com and nike.net webpages. The input is a product code, and the output is what Nike calls 2-line-name (which are 2 lines that briefly describe the product. It can be 1stLine: *Nike Dunk Low Retro*, 2ndLine: *Men's Shoes,* the Product Details like: "*Shown: White/Photon Dust/Coconut Milk/Midnight Navy*", and the Feature and Benefits: "*Synthetic leather upper softens and gains vintage character with wear.".* All these were done manually by the marketers, and now, with their supervision, are done by the app. As of now, we have improved the copy generation from 22 per week per marketer to 160.

Stack: AWS, Anthropic, HuggingFace all-MiniLM-L6-v2 for embeddings, Jenkins

When the app receives a code, we search in our DB for what we call the *raw name*, which is a brief description of the product, but acts like a code. This raw name is defined by the marketing team. This raw name can be something similar to: "M NK DNK LW R". Once we have that, we search for similar raw names in a curated database. With embeddings, we take the most similar to those, and we use those ones as examples to enrich the prompt. The prompt is also enriched with some descriptions or details that are saved in another DB.

We created several rules because sometimes some descriptions collide, so we need to adjust that as requested by the marketers. Example: for every shoe, if DNK appears in the raw name, it needs to be added in the First Line as Dunk, but if the shoe is a JORDAN one, it needs to be added in the 2nd line.

Architecture - RAG

Situation we have an app that marketeers use, whenever a user inputs a product code, it outputs Descriptions of the product, benefits, etc.These descriptions goes to **nike.com** and Nike.net. When the app receives a code, we search in our DB for what we call the *raw name*, which is a brief description of the product, but acts like a code. This raw name is defined by the marketing team. This raw name can be something similar to: "M NK DNK LW R". Once we have that, we search for similar raw names in a curated database. With embeddings, we take the most similar to those, and we use those ones as examples to enrich the prompt. The prompt is also enriched with some descriptions or details that are saved in another DB.

Task enrich with data/text from briefs used to present the products so that we can have an improved description for the products. Initially, they also wanted to have a RAG with vector search included

Actions

1. Define the stack.

2. Rethink if Vector Search is needed (make presentations to convince them that we did not need that) Although we kept that as a possibility
3. Redefine the stack
4. Define the way to save the data (by brief page, and not by product code to not duplicate) Created a presentation with the different options, pros and cons.
5. Confirm that the speed w 2 endpoints was fast enough (prod_id -> page numbers -> data from pages)
6. Created an MVP to showcase an example

Results the timing was decent enough, and everyone was happy about it. The idea is to start developing it at the beggining of the next quarter.

Description: architected a system to collect and store information from PDFs and presentations that marketers created when a new product is launched. This info is saved for later use to enrich the prompt in the product copy app generator. We also added a Vector DB so it can be used by the marketer to "talk" to these presentations and get information from different products at the same time (the original data is saved by product code)

Stack AWS, Databricks (Mosaic vector search), Online Tables, Airflow, ConnectorToBox

Challenges: The data collected from briefs needed to be saved to be retrieved by product code (since that is the input in our app). The retrieval needs to bring only the data relative to the product code. Sometimes in the briefs/presentations on one page, we may have similar data belonging to different codes, so we need to assign that data to the different codes. But if we are going to save the data for each code, then we would duplicate that data. Since we wanted the data to be retrieved almost instantly and we didn't want to duplicate data, we saved it by PDF page. We will have an intermediate table containing product-code -> pdf-page, for later retrieval of the pages needed for that product. The online tables are super fast in retrieval, so we don't spend a lot of time retrieving them. Also, with this, we don't duplicate the Vector DB Info.

XGBoost - SNKRS App

SNKRS app: it is a Nike App that offers shoes that are not for sale at stores at a regular price. These are shoes that you can only see in the SNKRS app. If you receive an offer, you won't receive another in the next 90 days, whether you redeemed it or not.

Stack: Databricks, Spark, AWS, Airflow, Jenkins, MLFlow, SKlearn

The model predicted the redemption rates for SNKRS app campaigns for all SNKRS app users (around millions). The input for the model was the

interactions with the app, Nike.com, and other apps (Nike Run and Nike Training App)

Initially, it was all run together (training + inference), and we needed to decouple into 2 different pipelines. Once we had this, we started training once a month and making the inference weekly. This move reduced the costs in clusters and did not impact the results. With the decoupling, we added hyperparameter tuning using Randomized Search. We tracked all the experiments in MLFlow.

Tell about the problem with the pipeline and not running the one-hot encoding in the same pipeline for inference.

NER AIRBUS - Datalogue

We needed to build an application that obfuscates the PII data in every dataset that Airbus owns. Our pipelines did not have that ability until then, and we needed to build a model to perform that task. We already had the infrastructure that was able to ingest data, process it (with our own tools), and then save it wherever the customer wants.

I believe we used Spacy, but I remember that at that time we needed to train the model (it was January 2020). So we needed to build a “tagger” so they could tag the words that the model needed to obfuscate later. Using that data, I think it was in HTML format. It was a really short project that I started in Argentina. While training and testing the model, I realized that the tagging wasn’t done properly. For example, they’ve tagged “New York” as two different words and not as a joint word. So it was over obfuscating some words. And it was not precise. The issue is that the tagging was very expensive, and was on their end, and we did not want them to do it again. So I needed to figure out a way to retag everything. I flew to Montreal for a week to meet the team and do it together (there were other areas involved in the project, although I was the only one in charge of the model). We were able to identify and re-tag everything back, retrain the model, and had better metrics. It was a Friday, and I went to the Montreal office with my suitcase ready to take a plane to Hartford. My sister lives there, and on Saturday it was her birthday. We had planned to celebrate it together. That morning, the CTO told me that I could come back on Monday to finish some of the infrastructure issues we were having. I think the obfuscation was not working. I had a flight back to Argentina the next Friday. I decided to stay, we moved the celebration to the next weekend, and we started working that same day. There was another backend developer who also came from New York to help us. We worked that whole weekend (at the CTO's apartment) and we finished everything by that Friday.

Stories:

7. Architecture pdfs/briefs databricks/openSearch

8. Soft delete conections at Datalogue
9. Discussion with Val and Nat regarding the RAG
10. XGBoost feature pipeline not dependent on training pipeline
11. NER for Airbus.
12. A/B Tests Jampp

Production AI System: Product Copy Generation with RAG

At Nike, I architected an end-to-end RAG system that generated product descriptions and marketing copy for Nike.com and Nike.net. Marketers would input a product code; the system retrieved a short internal descriptor (e.g., `M NK DNK LW R`), used embedding similarity to surface few-shot examples from a curated database, and injected those — alongside structured product attributes — into a prompt that an LLM used to generate the final copy.

The LLM sat at the very end of the pipeline. Most of the system's complexity lived upstream: retrieval logic, data modeling, and the ingestion pipeline I designed to bring in an entirely new data source — product launch briefs (PDFs and presentations) that marketers created for each new product.

The ingestion pipeline ran on Airflow, pulled files from Box, parsed and chunked them, and loaded the content into Databricks. The key architectural decision was how to store the brief data. Briefs are messy — a single page often contains information about multiple product codes, so naively indexing by product code would mean duplicating data across many rows. Instead, I stored content at the PDF page level and built an intermediate mapping table (`product_code → pdf_pages`). At inference time, two fast lookups via Databricks Online Tables retrieved only the pages relevant to that product. This kept the data clean, avoided duplication, and kept latency within acceptable bounds. The same page-level storage also powered a separate feature: a Mosaic Vector Search index that let marketers query across all briefs conversationally — effectively a chat interface over the entire product launch document corpus.

One thing I would do differently today: I'd invest earlier in a document parsing layer with layout awareness. We treated pages as flat text, but briefs are visual — tables, callout boxes, and image captions carry meaning that a naive text extraction loses. A more structured extraction step (using something like a vision model or a document-intelligence API) would have improved both retrieval precision and the quality of what the LLM actually received. That's the part of the pipeline I'd revisit first.